UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES

Sistemas Distribuidos

Clase 10 – Observabilidad distribuida

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Segundo cuatrimestre 2026

Equipo docente

2026

1 Introducción

Una vez que un sistema distribuido está desplegado en producción, el problema central deja de ser únicamente diseñarlo o codificarlo correctamente: pasa a ser también *entenderlo mientras opera*.

En una aplicación que ejecuta sobre un único servidor físico o máquina virtual (como un monolito tradicional en producción), el diagnóstico cuenta con sólidas garantías del sistema operativo y del hardware:

- **Stack trace completo:** el hilo de ejecución transcurre en un mismo espacio de direcciones de memoria. Si una excepción escapa, el runtime imprime el árbol de llamadas exacto, con archivo y número de línea.
- **Reloj físico unificado:** existe una única fuente de tiempo de hardware (TSC o cuarzo del procesador). Dos eventos registrados en el log local tienen una relación temporal estricta y verificable; el tiempo físico actúa como serializador de causalidad.
- **Llamadas en memoria:** la invocación entre funciones toma nanosegundos. Prácticamente todo el tiempo transcurre ejecutando código o esperando I/O local; no existen demoras invisibles de red.

Cuando el sistema se distribuye a través de múltiples servidores y microservicios comunicados por red, este modelo colapsa por cuatro razones estructurales de fondo:

1. **Destrucción del stack trace en la frontera de red:** El hilo de ejecución termina abruptamente en el socket TCP (`http.Client.Do` o llamada gRPC). El llamante solo observa un error genérico (`500 Internal Server Error` o `DeadlineExceeded`); la cadena causal queda mutilada y es imposible inferir qué falló cuatro saltos aguas abajo sin un mecanismo explícito de correlación.
2. **Pérdida del orden temporal por *clock drift*:** Entre distintos servidores no existe un reloj global confiable. Los desvíos de NTP superan a menudo la latencia de red. Si se combinan logs de múltiples nodos ordenándolos por timestamp, los eventos aparecen desordenados: la recepción de un mensaje puede figurar con una hora anterior a su despacho. El tiempo físico ya no sirve para deducir causalidad.
3. **Invisibilidad del tiempo "fuera del código":** En una red asincrónica, gran parte de la latencia no ocurre ejecutando lógica de negocio, sino esperando en colas intermedias: buffers de socket, colas de conexión en balanceadores, proxies y pools de workers. Un log local solo mide desde que el worker toma la solicitud; es ciego al tiempo de tránsito y encolamiento previo.
4. **Fallas combinatorias y acumulación de latencia de cola:** Las degradaciones en sistemas distribuidos rara vez son fallos binarios (up/down). Por el contrario, suelen ser el resultado de la acumulación de latencias a lo largo de la cadena o de combinaciones emergentes de atributos de negocio (por ejemplo, el caso del "Cliente Elefante", donde un tenant corporativo con carritos de 30 000 ítems genera micro-demoras en cada servicio downstream). En este escenario, **los dashboards y métricas locales de cada servidor individual se muestran 100% saludables (en verde)**, pero la suma acumulativa excede el timeout en el API Gateway y el usuario experimenta una falla total.

Por estas razones, la observabilidad no es un agregado opcional al final del desarrollo. Es una **propiedad arquitectónica estructural** del sistema.

2 ¿Por qué monitorear?

Antes de hablar de herramientas o de la definición formal de observabilidad, conviene responder una pregunta más básica: ¿por qué invertir esfuerzo en monitoreo?

En un sistema local, un error suele ser visible de inmediato: una excepción, un crash, un test que falla. En un sistema distribuido, las fallas pueden ser **parciales, intermitentes y silenciosas**.

Las razones fundamentales para monitorear son:

Detección temprana: Hay problemas que no se resuelven de manera automática y es necesario informar a un humano: un disco que se está llenando, un certificado por vencer, un servicio que degrada lentamente. Sin monitoreo, estos problemas se descubren cuando ya es demasiado tarde (típicamente por quejas de usuarios).

Diagnóstico: Poder reconstruir qué pasó y por qué, no solo constatar que algo falló.

Decisiones informadas: Sin datos operativos, las decisiones de capacidad, despliegue y priorización son apuestas a ciegas. Por ejemplo: ¿cuánto mejoró el hit rate de memcache al agregar un nodo nuevo? ¿Justifica el costo de infraestructura? Sin métricas, no hay forma de saberlo.

Contención de daño: Un sistema que detecta su propia degradación puede activar mecanismos de resiliencia (circuit breakers, graceful degradation, rate limiting) antes de que el daño se propague en cascada.

Aprendizaje continuo: Los datos de monitoreo alimentan postmortems blameless, experimentos de caos (Chaos Engineering) y mejoras continuas de diseño arquitectónico.

Esta taxonomía es de la cátedra. El *SRE Book* (Beyer et al., 2016, cap. 6) organiza las razones de otro modo: análisis de tendencias de largo plazo, comparación entre períodos o grupos experimentales, *alerting*, construcción de dashboards y análisis retrospectivo *ad hoc*. El ejemplo del *hit rate* de memcache está tomado de allí.

3 Observabilidad: De la Teoría de Control al Software

3.1 Definición conceptual

La formulación canónica, heredada de la Teoría de Control (Rudolf E. Kálmán, 1960), establece que:

> Un sistema es observable si su estado interno puede inferirse razonablemente a partir del conocimiento de sus salidas externas.

Aplicado a la ingeniería de software moderna (Majors et al., 2022; Young & Parker, 2024), un sistema es observable si permite a los ingenieros investigar y explicar cualquier estado arbitrario en el que se encuentre el sistema —especialmente estados imprevistos o nunca antes vistos (*unknown-unknowns*)— mediante preguntas exploratorias sin necesidad de adivinar, predecir el fallo de antemano o desplegar nuevo código para agregar más logs.

3.2 La trampa de los "Tres Pilares" vs. "La Trenza Única de Datos"

Tradicionalmente, la industria ha presentado la observabilidad como los "tres pilares": **logs, métricas y trazas**. Sin embargo, la literatura moderna advierte que esta visión es engañosa (Young & Parker, 2024).

Construir logs, métricas y trazas como tres sistemas verticales aislados produce el antipatrón conocido como **"Las Tres Pestañas del Navegador"** (*Three Browser Tabs of Observability*):

- Una pestaña con dashboards de métricas agregadas (Prometheus/Grafana).
- Otra pestaña con buscadores de logs de texto libre (ELK/OpenSearch).
- Una tercera pestaña con herramientas de tracing (Jaeger/Zipkin).

El problema de las tres pestañas es que la información vive en silos inconexos. Cuando ocurre un incidente, el operador se ve forzado a hacer de puente manual, memorizando IDs y saltando entre pestañas intentando correlacionar eventos a simple vista.

La observabilidad moderna trasciende las tres pestañas mediante **"La Trenza Única de Datos"** (*Single Braid of Data*): las señales de telemetría (logs, métricas, trazas) conservan sus propósitos especializados, pero están **interconectadas y entrelazadas estructuralmente mediante contexto compartido** (OpenTelemetry Protocol - OTLP).

3.3 Hard Context vs. Soft Context

Para que las herramientas puedan calcular correlaciones automáticas en la trenza de datos, es vital distinguir entre dos tipos de contexto:

- **Soft Context (Contexto Blando):** Atributos que cada instrumento adjunta a la medición: identificador de cliente, hostname del balanceador, *timestamp*. Pueden acompañar una relación causal, pero no la garantizan. Su valor es que **definen las dimensiones** por las que se puede cortar y agrupar la telemetría: descubrir que el 98 % de los fallos comparten una misma `availability_zone` es trabajo de soft context. El soft context más usado en monitoreo es el tiempo, y es también el más débil: cuando es la única dimensión compartida, el operador termina alineando marcas temporales a ojo.
- **Hard Context (Contexto Duro):** Enlaces explícitos, estructurales e inmutables inyectados a nivel de protocolo (`trace_id`, `span_id`, `parent_span_id`, cabeceras W3C Trace Context). Define la topología causal exacta de una solicitud individual, permitiendo conectar logs y métricas (vía *exemplars*) directamente con su traza correspondiente. **Define la forma** del sistema: qué se relaciona causalmente con qué.

En síntesis (Young & Parker, 2024, cap. 2): el hard context define la *forma* del sistema y el soft context define las *dimensiones* que explican qué representa cada señal. Lo frágil no es el soft context —sin él no hay análisis diferencial posible—, sino el soft context *sin* hard context.

3.4 La arquitectura de la Trenza Única: OTLP, Collector y Correlación Automática

Para comprender cómo la "Trenza Única de Datos" supera en la práctica el aislamiento de los tres pilares tradicionales, es necesario examinar su implementación técnica en dos niveles: la unificación del transporte y los mecanismos de correlación automática calculados por software.

3.4.1 Unificación de transporte: De la proliferación de agentes a OTLP

Históricamente, incorporar telemetría completa a un servicio exigía instalar y mantener múltiples bibliotecas y procesos demonio en el host:

- un cliente de métricas (ej: cliente de Prometheus) que exponía un puerto de *scraping*;
- una biblioteca de logs que volcaba texto a disco y un agente secundario (ej: Filebeat o Fluentd) que parseaba los archivos con expresiones regulares;

- un SDK de tracing especializado (ej: Jaeger o Zipkin) que enviaba spans por UDP en formatos binarios propietarios.

Esta proliferación generaba sobrecarga de memoria en la aplicación, competencia por recursos y un fuerte acoplamiento: cambiar de proveedor de almacenamiento obligaba a reescribir código de instrumentación.

El estándar OpenTelemetry (Young & Parker, 2024) resuelve esta fragmentación mediante una arquitectura desacoplada:

1. **Un único SDK en la aplicación:** El código del servicio se instrumenta contra una API neutral y unificada, sin importar qué backend de almacenamiento se utilice.
2. **Un protocolo común (*OpenTelemetry Protocol* - OTLP):** Un único canal de red para emitir métricas, logs y trazas en lotes (*batches*). OTLP admite **tres transportes**: `grpc`, `http/protobuf` y `http/json`. Los dos primeros serializan con Protocol Buffers; el tercero, con JSON. El transporte por defecto y recomendado es `http/protobuf` (Young & Parker, 2024), no gRPC, pese a que gRPC sea el más asociado al proyecto. Puertos por defecto: 4317 para gRPC y 4318 para HTTP.
3. **El OpenTelemetry Collector:** Un proceso intermediario independiente (desplegado típicamente como *sidecar* o agente de nodo) que recibe telemetría en formato OTLP, aplica transformaciones, filtros o muestreo, y la encamina concurrentemente hacia múltiples destinos (Prometheus, Jaeger, Elasticsearch, etc.) mediante configuración declarativa, sin intervención del código de la aplicación.

3.4.2 Mecanismos de correlación automática por software

Afirmar que las computadoras calculan correlaciones automáticas significa que el análisis entre métricas, logs y trazas deja de basarse en la memoria visual del operador y pasa a operar mediante mecanismos determinísticos:

1. **Contexto activo en memoria:** Al recibir una solicitud, el framework de instrumentación aloja en la memoria del hilo o corutina (`Context` en Go, `ThreadLocal` en Java) los identificadores inmutables de la traza activa (`trace_id` y `span_id`).
2. **Correlación Log–Trace (Inyección automática de metadatos):** Cuando el desarrollador invoca el logger de la aplicación, el SDK intercepta la llamada, extrae automáticamente el `trace_id` y `span_id` del contexto activo y los estampa como campos estructurales del evento JSON. Como consecuencia, las herramientas de análisis pueden ejecutar un *JOIN* relacional exacto entre un span y todos los logs generados durante su ejecución, eliminando las búsquedas manuales por aproximación temporal.
3. **Correlación Métrica–Trace (*Exemplars*):** En sistemas de métricas basados en series temporales (como Prometheus y el estándar OpenMetrics), los histogramas agrupan la latencia en cubos (*buckets*) agregados. Un **Exemplar** es una muestra representativa que asocia el `trace_id` de una petición individual a un punto de datos específico dentro del histograma. En los paneles de control (Grafana), esto se visualiza como un enlace directo sobre la curva de latencia: al hacer clic en un pico del percentil p99, la interfaz abre de inmediato la traza individual que generó esa degradación.
4. **Análisis diferencial multidimensional (*BubbleUp*):** Dado que métricas, trazas y eventos anchos comparten las mismas etiquetas de infraestructura y de negocio (`Resource Attributes`), el motor de observabilidad puede comparar estadísticamente la distribución de todos los atributos entre las solicitudes que fallaron y las que operaron con éxito, aislando cuantitativamente la dimensión causante de la anomalía sin requerir hipótesis previas.

4 ¿Qué debe responder un buen monitor?

Las dos preguntas fundamentales que cualquier sistema de telemetría y monitoreo debe responder son:

1. **¿Qué está roto?** — El monitor debe poder señalar con razonable precisión qué componente, servicio o recurso está degradado. "El sistema está lento" no es una respuesta útil; "el servicio de pagos tiene latencia p99 $>$ 2s" sí lo es.
2. **¿Por qué está roto?** — El monitor debe proporcionar suficiente contexto para iniciar el diagnóstico: ¿saturación de CPU? ¿dependencia downstream caída? ¿cambio de configuración reciente? ¿un cliente específico saturando el ancho de banda?

Este principio tiene consecuencias directas sobre el diseño de la trenza de datos:

- Las **métricas** son ideales para responder rápidamente "qué está roto".
- Las **trazas** conectan el "qué" con el "por qué" a través del camino distribuido.
- Los **eventos estructurados** aportan el detalle fino de dominio para aislar la causa raíz del "por qué".

5 Logs: De Texto Libre a Eventos Estructurados

5.1 Qué responden

Los logs responden a la pregunta: *¿Qué ocurrió en un instante específico de la ejecución?*

5.2 Texto libre vs. Logs estructurados

Un log de texto libre (por ejemplo, `"User 42 logged in from 10.0.0.1"`) es legible para humanos pero inadecuado para procesamiento distribuido, ya que exige expresiones regulares frágiles ante cambios de formato.

En sistemas distribuidos se exige **log estructurado** (típicamente JSON) con campos bien definidos:

- `timestamp` de alta precisión, en UTC y con formato único en todo el sistema;
- `service_name` y `service_version`;
- `trace_id` y `span_id` (**Hard Context**);
- `severity_text` (INFO, WARN, ERROR);
- `attributes` (diccionario de clave-valor de dominio).

5.3 La unidad de trabajo: Eventos Estructurados Anchos

La tendencia moderna es agrupar la información de una unidad de trabajo en un **Evento Estructurado Ancho** (*Arbitrarily Wide Structured Event*, Majors et al., 2022). En lugar de emitir múltiples líneas sueltas durante un pedido, el servicio inicializa un mapa en memoria al recibir la solicitud, acumula metadatos durante su ejecución y emite un único evento estructurado rico al finalizar.

6 Métricas: Agregaciones, Series Temporales y Percentiles

6.1 Qué responden

Las métricas responden a la pregunta: *¿Cuál es la tendencia o salud agregada del sistema a lo largo del tiempo?*

6.2 Golden Signals

Las cuatro señales doradas (Google SRE, 2016) constituyen la base del monitoreo de servicios:

- **Latencia:** Tiempo que toma procesar una solicitud (diferenciando éxito de error).
- **Tráfico:** Demanda sobre el sistema (ej: solicitudes HTTP/segundo).
- **Errores:** Tasa de solicitudes que fallan (explícitas como HTTP 500, implícitas como payload incorrecto).
- **Saturación:** Qué tan lleno está el recurso más restringido (memoria, pool de conexiones de DB, CPU). Dos advertencias del propio SRE Book que suelen olvidarse: muchos sistemas se degradan *antes* de alcanzar el 100 % de utilización, por lo que fijar un objetivo de utilización es imprescindible; y el aumento de latencia suele ser un **indicador adelantado** de saturación —medir el percentil 99 sobre una ventana corta da una señal temprana—. Esto enlaza esta señal con la sección de percentiles.

6.3 La falacia del promedio: Percentiles vs. Media

En sistemas distribuidos, el promedio de latencia oculta las anomalías de la cola larga. Si 99 usuarios tardan 10 ms y 1 usuario tarda 10 000 ms, el promedio es 110 ms, ocultando que un usuario sufrió una degradación inaceptable.

Se debe medir y alertar siempre sobre percentiles:

- **p50 (Mediana):** Experiencia típica del usuario.
- **p95 / p99:** Comportamiento de la cola larga bajo estrés o contención de bloqueos.

6.4 El peligro de la Explosión de Cardinalidad en TSDBs

Una base de datos de series temporales (TSDB como Prometheus) crea una serie temporal por cada combinación única de etiquetas (dimensiones).

Si se inyecta un campo con alta cardinalidad (como `user_id` con 10 millones de usuarios o `order_id`) en una métrica:

$$\text{Total series} = \text{métricas} \times \text{métodos} \times \text{status_codes} \times \text{user_ids}$$

Cada combinación de etiquetas es una serie temporal residente, de modo que agregar una dimensión **multiplica** la cantidad de series y con ella la memoria (**Explosión de Cardinalidad**). El crecimiento es combinatorio en la cantidad de dimensiones —como muestra el producto de arriba—; llamarlo "exponencial" es impreciso.

> **Regla de diseño fundamental:** Las métricas agregadas solo deben contener etiquetas de baja cardinalidad finita (`http_method`, `status_code`, `endpoint`). La alta cardinalidad pertenece a los Spans y Eventos Estructurados.

7 Tracing Distribuido y Causalidad

7.1 Qué responde

El tracing distribuido responde a la pregunta: *¿Cómo atravesó esta solicitud la red a través de los múltiples servicios?*

7.2 Conceptos fundamentales

- **Trace:** El viaje completo de una solicitud a través de la topología distribuida, modelado como un Grafo Acíclico Dirigido (DAG).
- **Span:** La unidad individual de trabajo delimitada en tiempo dentro de un trace (ej: llamada RPC, consulta SQL o cómputo interno).

Un Span contiene cinco elementos obligatorios: Trace ID, Span ID, Parent Span ID (ausente en el root span), Timestamp de inicio y Duración.

7.3 Semántica y roles de un Span: SpanKind

Un span no es únicamente un intervalo de tiempo genérico. Sin conocer el rol del componente que lo emite, las herramientas de observabilidad no pueden distinguir si un intervalo prolongado responde a cómputo intensivo local, espera de red o encolamiento asincrónico.

Para resolver esto, la *OpenTelemetry Specification* (API de trazas) define el atributo `SpanKind`, que establece el rol semántico del span dentro de la topología distribuida:

`CLIENT`**:** Representa una llamada síncrona saliente hacia un servicio remoto o recurso externo (por ejemplo, una solicitud HTTP saliente, una invocación gRPC o una consulta SQL a una base de datos). El span se inicia al comenzar a preparar el envío y finaliza cuando se recibe la respuesta completa.

`SERVER`**:** Representa la recepción y atención de una solicitud síncrona entrante en el lado del servidor (por ejemplo, un handler HTTP o un método de servicio gRPC). Inicia al recibir el pedido y finaliza cuando la respuesta es despachada al cliente.

`PRODUCER`**:** Modela el despacho asincrónico de un mensaje hacia un broker o intermediario de mensajería (como Kafka, RabbitMQ o AWS SQS). A diferencia de un span `CLIENT`, finaliza tan pronto como el broker confirma la recepción del mensaje, sin esperar a que un consumidor lo procese.

`CONSUMER`**:** Modela la recepción y el procesamiento asincrónico de un mensaje extraído de una cola o tópico. A diferencia de `SERVER`, el procesamiento ocurre de forma desacoplada en el tiempo respecto del despacho del productor.

`INTERNAL`**:** Modela operaciones internas dentro del mismo proceso o servicio que no cruzan límites de red ni interactúan con colas externas (por ejemplo, un cálculo matemático complejo, un algoritmo de renderizado o procesamiento en memoria).

Aislamiento de latencia de red (par CLIENT–SERVER): En llamadas síncronas entre dos servicios instrumentados con OpenTelemetry, el span `CLIENT` del llamante y el span `SERVER` del receptor forman un par: el `SERVER` es hijo del `CLIENT` y, en el caso normal, queda contenido temporalmente dentro de él. No siempre: si el llamante abandona por *timeout*, el servidor puede seguir trabajando después de que el span `CLIENT` ya cerró.

La resta duración(`CLIENT`) − duración(`SERVER`) acota el tiempo que **no** transcurrió dentro del handler del servidor: red de ida, red de vuelta y encolamiento previo a que el handler tome la solicitud. Es una *suma* de los tres; la resta no permite separarlos. Su virtud es que no exige relojes sincronizados, porque cada duración se mide contra un único reloj local.

Nota histórica: el modelo de Dapper era distinto. Dapper (Sigelman et al., 2010, §2.1) no usaba dos spans por RPC sino **uno solo, compartido por ambos hosts**: *"in a typical Dapper trace we expect to find a single span for each RPC"* y *"every RPC span contains annotations from both the client and server processes, making two-host spans the most common ones"*. Con un span de dos hosts, cliente y servidor escriben timestamps de relojes distintos en el mismo registro, y Dapper trataba el desfase por acotación: como el cliente siempre envía antes de que el servidor reciba (y viceversa para la respuesta), se obtienen una cota inferior y una superior para los timestamps del lado servidor. El paso al par `CLIENT`/`SERVER` de OpenTelemetry es una decisión de modelo de datos con consecuencias medibles: cada extremo mide con su propio reloj y la comparación se hace después, en el análisis.

El límite de la jerarquía simple: Span Links: En flujos síncronos, la causalidad se expresa limpiamente como un árbol jerárquico mediante `parent_span_id`. Sin embargo, en arquitecturas basadas en eventos con procesamiento por lotes (*batch processing*), un trabajador `CONSUMER` puede procesar simultáneamente múltiples mensajes provenientes de trazas independientes. Dado que un span solo admite un único padre directo, el modelo incorpora **Span Links** (*enlaces entre spans*): referencias causales que conectan el span actual con múltiples trazas antecedentes sin forzar una jerarquía estricta de padre e hijo.

7.4 Conexión teórica con la Clase 03: Causalidad de Lamport vs. Clock Drift

En la Clase 03 estudiamos los Relojes Lógicos de Lamport y la relación *happened-before* ($\rightarrow$). En sistemas distribuidos no podemos confiar en el tiempo físico para ordenar eventos debido al desvío de los relojes de hardware (*clock drift/skew*).

Un Trace no es una simple lista ordenada por tiempo físico. Un Trace es la materialización del modelo de causalidad de Lamport. El campo `parent_span_id` establece la relación causal estricta $Span_A \rightarrow Span_B$ independientemente de los desfases en los relojes de las máquinas.

7.5 Propagación de Contexto (In-Band vs. Out-of-Band)

Basado en el modelo Dapper (Sigelman et al., 2010):

- **Propagación In-Band (En banda):** El contexto de la traza (`traceparent` bajo estándar W3C: `00-trace_id-parent_id-flags`) viaja **dentro de los mensajes de red** (cabeceras HTTP, metadatos gRPC). Si un servicio no reinyecta estas cabeceras al invocar aguas abajo, la cadena causal se quiebra.
- **Recolección Out-of-Band (Fuera de banda):** Al completarse un Span, sus datos se transmiten asíncronamente a un recolector (OpenTelemetry Collector) mediante buffers en memoria para no penalizar la latencia del usuario.

8 Estrategia de Instrumentación en Dos Capas

Para lograr observabilidad sostenible, se recomienda una estrategia en dos capas:

- **Capa 1: Infraestructura de Propagación (Auto-instrumentación / Framework)**
  Mediante agentes o middlewares automáticos (OpenTelemetry), se captura el esqueleto del sistema: spans raíz, propagación de cabeceras W3C y latencias de transporte. No requiere modificar lógica de negocio.
- **Capa 2: Enriquecimiento Semántico de Dominio (Instrumentación Manual)**
  El desarrollador inyecta atributos de negocio en los spans activos (por ejemplo: `tenant.id` o `payment.method`). Otorga significado semántico a la traza para responder preguntas de negocio en producción.

9 Debugging en Producción y el Core Analysis Loop

9.1 Por qué la causa raíz no coincide con el síntoma visible

En arquitecturas distribuidas, las fallas se propagan en cascada:

1. Una base de datos downstream se degrada por contención de bloqueos.
2. El servicio intermedio agota su pool de hilos encolando peticiones.
3. Los clientes upstream reintentan automáticamente, amplificando el tráfico.
4. El API Gateway de borde comienza a retornar errores HTTP 504.

El síntoma visible estalla en el borde, pero la causa raíz reside aguas abajo.

9.2 El Core Analysis Loop (Análisis por Primeros Principios)

En lugar de depender de la intuición o de dashboards estáticos, la observabilidad propone un flujo iterativo y estructurado (*Core Analysis Loop*, Majors et al., 2022):

1. **Observar la anomalía agregada (Métrica):** un salto en el percentil p99 de latencia o en la tasa de errores. Indica *que* algo cambió, no *qué*.
2. **Diferenciar dimensiones (Eventos estructurados anchos):** computar qué atributo separa las solicitudes anómalas de la línea base. Por ejemplo, `availability_zone=us-east-1a` concentra el 98 % de los fallos frente al 15 % usual. Es el *soft context* el que hace posible este corte.
3. **Confirmar con una traza individual (Tracing):** abrir una solicitud representativa del subconjunto anómalo y determinar en qué span se consumió el tiempo.

Si el atributo hallado no explica suficientemente la anomalía, se filtra la vista por ese atributo y se vuelve al paso 2 con el subconjunto reducido. Esa iteración es lo que convierte el procedimiento en un *loop* y no en una lista de pasos: es un método de fuerza bruta que recorre las dimensiones disponibles sin exigir conocimiento previo del sistema. Recorrerlas a mano resulta impracticable a escala, de modo que las herramientas de observabilidad automatizan el paso 2 comparando todas las dimensiones dentro y fuera del área anómala y ordenándolas por discrepancia (Majors et al., 2022, cap. 8).

10 Modelo del Curso

Expresamos el problema operacional de la observabilidad mediante el framework del curso:

$$D = (P, F, G, S, T)$$

- P **(Problema):** Diagnosticar y entender el comportamiento de un sistema distribuido en producción ante fallas imprevistas.
- F **(Factores/Restricciones):** Ausencia de reloj global (*clock drift*), fallas en cascada, latencia variable de red, explosión de cardinalidad.
- G **(Garantías/Objetivos):** Reconstrucción de la causalidad distribuida, detección de causas raíz en minutos, preservación del valor de negocio.
- S **(Solución):** Trenza única de datos (OTEL/OTLP) + Propagación de contexto duro (W3C) + Instrumentación en 2 capas + Debugging por primeros principios.
- T **(Trade-offs/Costos):** Sobrecarga de cómputo y red por telemetría, costo de almacenamiento para alta cardinalidad, complejidad de configuración.

11 Conclusión

No es posible operar confiablemente un sistema distribuido sin diseñar su observabilidad desde el inicio. Entrelazar logs, métricas y trazas mediante propagación de contexto transforma la complejidad del sistema en una estructura navegable y comprensible.

> Observabilidad distribuida no es "mirar dashboards ni abrir tres pestañas". Es diseñar el software para que transporte contexto causal y pueda explicarse a sí mismo cuando falle en producción.

12 Síntesis razonada de la clase

Esta unidad debe leerse como parte de una cadena de decisiones de diseño y no como un catálogo de mecanismos. Para estudiar el tema con profundidad conviene reconstruir cuatro preguntas: *qué problema aparece*, *qué garantía se desea*, *qué mecanismo la aproxima* y *qué costo introduce*. En cada ejemplo debe identificarse además el modelo de falla implícito.

> Una respuesta técnicamente madura no termina al nombrar un algoritmo o patrón: declara sus supuestos, la propiedad que preserva y el escenario en el que deja de ser suficiente.

13 Bibliografía guiada y ruta de profundización

Para profundizar esta clase se recomienda: *Site Reliability Engineering* (cap. 6) para señales y monitoreo; *Learning OpenTelemetry* (caps. 1 a 3) para el modelo conceptual, el contexto duro y OTLP; *Observability Engineering* (caps. 5 y 8) para eventos anchos y análisis; y el paper de Dapper para el origen del tracing distribuido.

Referencias canónicas

1. **Young, T. y Parker, A.** (2024). *Learning OpenTelemetry: Setting Up and Operating a Modern Observability System*. O'Reilly Media. (Capítulos 1, 2 y 3).
2. **Majors, C., Fong-Jones, L. y Miranda, G.** (2022). *Observability Engineering: Achieving Production-Excellence*. O'Reilly Media. (Capítulos 1, 5, 6 y 8; el *core analysis loop* está en el 8).

3. **Sigelman, B. H., et al.** (2010). *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google Technical Report.
4. **Beyer, B., et al.** (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. (Capítulo 6, *Monitoring Distributed Systems*).
5. **Kálmán, R. E.** (1960). *On the General Theory of Control Systems*. IFAC Proceedings Volumes, 1(1), 491–502.
6. **OpenTelemetry Authors.** *OpenTelemetry Specification*: API de trazas (`SpanKind`, *Links*) y especificación de OTLP. `https://opentelemetry.io/docs/specs/`

La lectura debe hacerse con preguntas concretas: (1) ¿qué modelo de sistema asume el autor?, (2) ¿qué propiedad demuestra o persigue?, (3) ¿qué falla o anomalía motiva el mecanismo?, y (4) ¿qué trade-off queda abierto?

14 Conexión con el Trabajo Final Integrador

El grupo debe señalar qué parte de su sistema utiliza los conceptos de esta unidad, justificar por qué son necesarios y documentar una alternativa descartada. Cuando el contenido todavía no corresponda a la etapa vigente, debe registrarse como hipótesis o decisión pendiente.

15 Autoevaluación conceptual

1. ¿Cuál es el problema distribuido concreto que motiva esta clase?
2. ¿Qué propiedad de seguridad y qué propiedad de progreso están en juego?
3. ¿Qué supuesto de red, tiempo o falla resulta decisivo?
4. ¿Qué trade-off aceptaría en un sistema real y cuál no aceptaría en un dominio crítico?
5. ¿Cómo observaría experimentalmente que la solución se comporta como espera?

16 Profundización autocontenida v9

16.1 Problema, límite e invariante

El núcleo de esta clase es logs estructurados, métricas, tracing, propagación de contexto, percentiles y debugging causal. Para estudiarlo de manera autónoma conviene separar tres planos. Primero, el *problema observable*: qué comportamiento incorrecto puede ver un cliente o un operador. Segundo, el *límite del modelo*: qué información, sincronía o coordinación no está disponible. Tercero, el *invariante*: qué propiedad no debe violarse aunque el sistema pierda progreso temporalmente.

El modelo de fallas relevante incluye telemetría incompleta, cardinalidad, sampling, relojes distintos y causa upstream oculta. Una solución debe describirse como protocolo y no solo como nombre de tecnología: actores, estados, mensajes, condición de éxito, condición de aborto y estado visible después de una interrupción. La evidencia esperada es traza extremo a extremo, logs correlacionados y métricas p50/p95/p99.

16.2 Método de análisis

Ante un caso nuevo, siga esta secuencia:

1. Identifique actores, dato o recurso compartido y frontera de confianza.
2. Escriba una ejecución nominal y una ejecución adversarial.
3. Distinga seguridad (lo malo no ocurre) de progreso (algo bueno finalmente ocurre).
4. Declare qué supuesto permite avanzar: mayoría, deadline, sincronía parcial, operación idempotente, almacenamiento durable u otro.
5. Compare una alternativa más simple y una más fuerte; registre qué métrica o experimento haría cambiar la decisión.

16.3 Caso transferible

Considere una plataforma de reservas que recibe operaciones duplicadas, pierde conectividad entre regiones y debe sostener un camino crítico sin ocultar el estado real al usuario. Aplique el mecanismo de esta clase a una única decisión concreta. Una respuesta madura no intenta resolver todo: delimita la operación, formula la garantía y explica la degradación esperada cuando el supuesto central deja de cumplirse.

16.4 Errores de razonamiento frecuentes

- confundir una propiedad con el producto que suele implementarla;
- describir el camino feliz sin recorrer una falla intermedia;
- afirmar “exactly-once”, “alta disponibilidad” o “consistencia” sin definir alcance observable;
- medir solo throughput aunque la hipótesis trate sobre semántica o recuperación;
- presentar la complejidad añadida como beneficio sin compararla con la alternativa más simple.

17 Lectura guiada

Google SRE; OpenTelemetry; USE/RED methods y cardinalidad de métricas.

Durante la lectura, registre: modelo de sistema, propiedad perseguida, contraejemplo que motiva el mecanismo, costo principal y una condición bajo la cual no lo elegiría. Estas cinco anotaciones convierten la bibliografía en una herramienta de diseño y no en una lista para memorizar.

18 Aplicación al Trabajo Final Integrador

El equipo debe producir un ADR breve con la decisión asociada a esta clase. Debe contener: contexto, dos alternativas, supuesto decisivo, garantía elegida, trade-off, evidencia pendiente y fecha de revisión. La evidencia mínima esperada para esta unidad es: traza extremo a extremo, logs correlacionados y métricas p50/p95/p99.

19 Autoevaluación avanzada

1. ¿Puedo explicar la clase sin nombrar primero una tecnología?
2. ¿Puedo construir una ejecución que viole la garantía si el mecanismo se configura mal?
3. ¿Puedo distinguir seguridad, progreso y experiencia del usuario?

4. ¿Puedo proponer una medición que valide la propiedad, no solo el rendimiento?
5. ¿Puedo defender cuándo una solución más simple sería suficiente?